# Solutions to selected exercises - Fitting ODE models to data

Katie Montovan

2025-08-21

These are solutions to the exercises where it might be helpful to see an example of the R code if you get stuck. These are not solutions to all the exercises.

**Exercise 5:** Calculate $I(1), I(1.5)$, and $I(2)$. We use a vector $I$ to store these values.

```
I=4
Beta=0.0001
N=1000
gamma=0.15
deltaT=0.5

#Calculate I(0.5)
I[2]=I[1]+deltaT*((Beta*N-gamma)-Beta*I[1])*I[1]

#Calculate I(1)
I[3]=I[2]+deltaT*((Beta*N-gamma)-Beta*I[2])*I[2]

#Calculate I(1.5)
I[4]=I[3]+deltaT*((Beta*N-gamma)-Beta*I[3])*I[3]

#Calculate I(2)
I[5]=I[4]+deltaT*((Beta*N-gamma)-Beta*I[4])*I[4]

I
```

```
## [1] 4.000000 3.899200 3.800960 3.705213 3.611897
```

**Exercise 6:** Using R or a spreadsheet use the Euler method with $\Delta t = 1$ and $\beta = .0001$ to calculate the predicted number in $I$ for $t = 1, 2, 3, \ldots, 14$

Setup a data frame with the data that we were given:

```
data=data.frame(Day = 0:14,I =  c(4, 3 , 5 , 6 , 8 , 15 , 13 , 18 , 24 , 37, 60, 82, 110, 141, 187))
data
```

```
##    Day   I
## 1    0   4
## 2    1   3
## 3    2   5
## 4    3   6
## 5    4   8
## 6    5  15
## 7    6  13
## 8    7  18
## 9    8  24
## 10   9  37
```

```
## 11  10  60
## 12  11  82
## 13  12 110
## 14  13 141
## 15  14 187
```

Estimate I if $\beta = 0.0001$ and graph the result with the data.

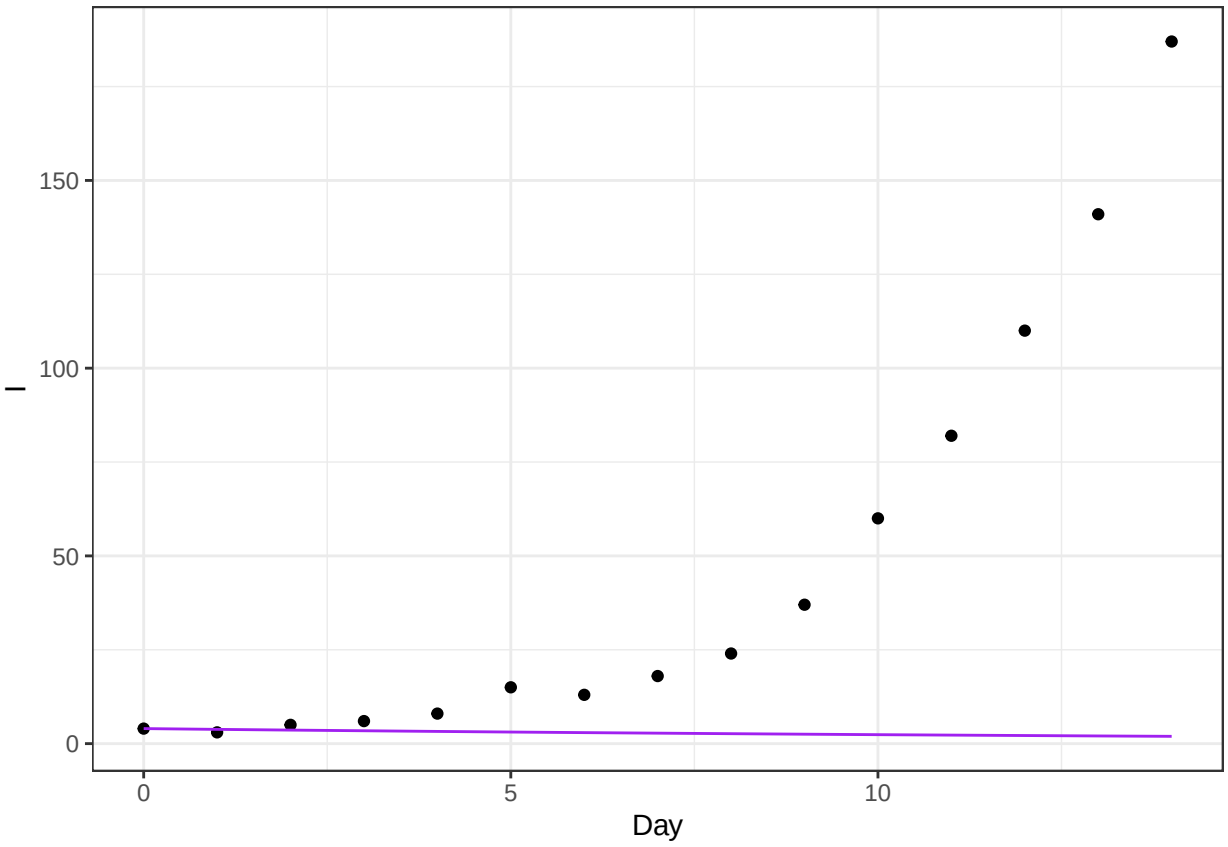
```
I1=4
Beta=0.0001
N=1000
gamma=0.15
deltaT=1

for(j in 2:15){
  I1[j]=I1[j-1]+deltaT*((Beta*N-gamma)-Beta*I1[j-1])*I1[j-1]
}
I1
```

```
##  [1] 4.000000 3.798400 3.607037 3.425384 3.252942 3.089236 2.933820 2.786269
##  [9] 2.646179 2.513170 2.386880 2.266966 2.153104 2.044985 1.942317
```

```
data|>
  mutate(I_0001=I1) -> data
```

```
ggplot(data,aes(x=Day,y=I))+
  geom_point(aes(x=Day,y=I),col="black")+
  geom_line(aes(y=I_0001),col="purple")+
  theme_bw()
```



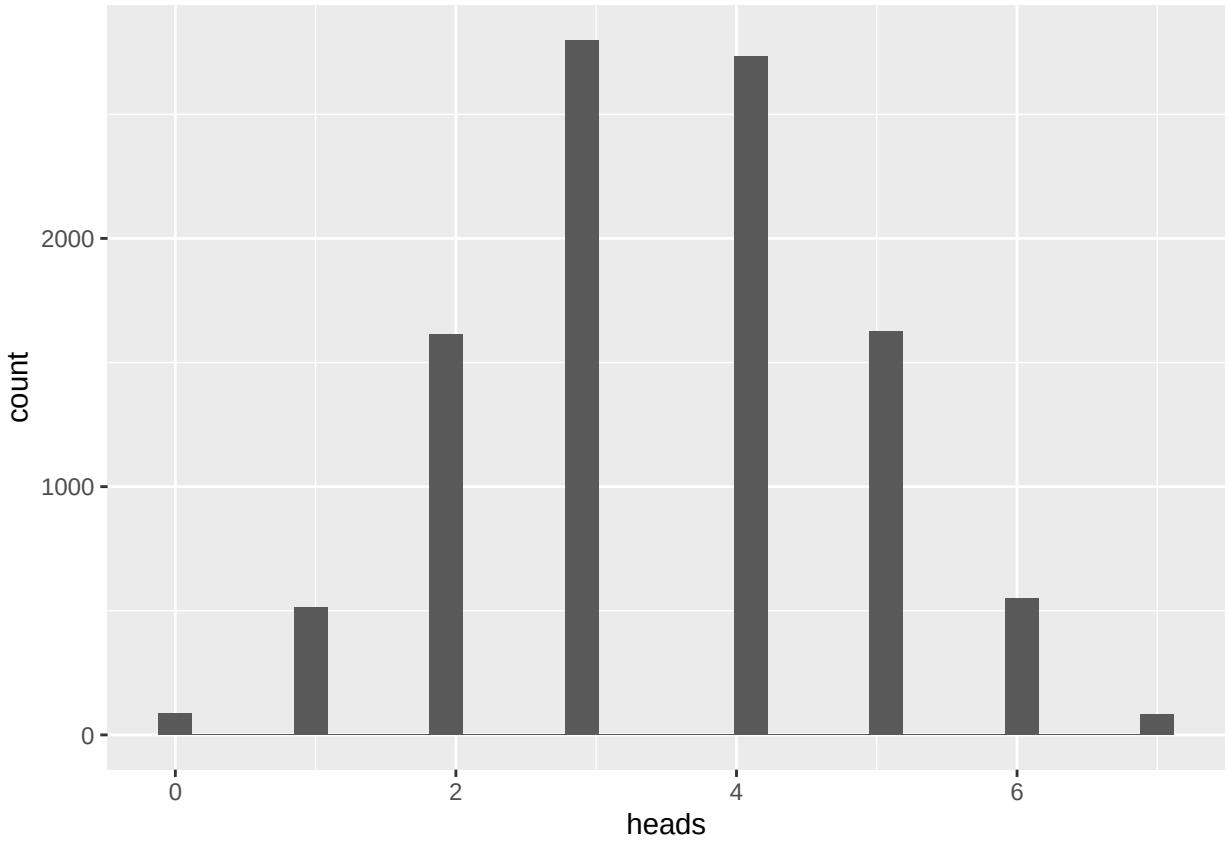
We see the model (purple) under-predicts the number of infections (data is the black dots). From this we would expect a model with a larger $\beta$ value to do better. We leave the joy of furster exploration to you!

**Exercise 8, parts:** For this exercise we will flip 7 fair coins. In R, use the **rbinom** function to simulate 10,000 trials, create a histogram of the number of heads, and tally the number of times each count occurs.

```
flips = rbinom(10000,7,.5)

flip_data=data.frame(heads=flips)
ggplot(data=flip_data,aes(x=heads))+
  geom_histogram()
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```




```
flip_data|>
  group_by(heads)|>
  summarise(count=n(),freq=count/10000)
```

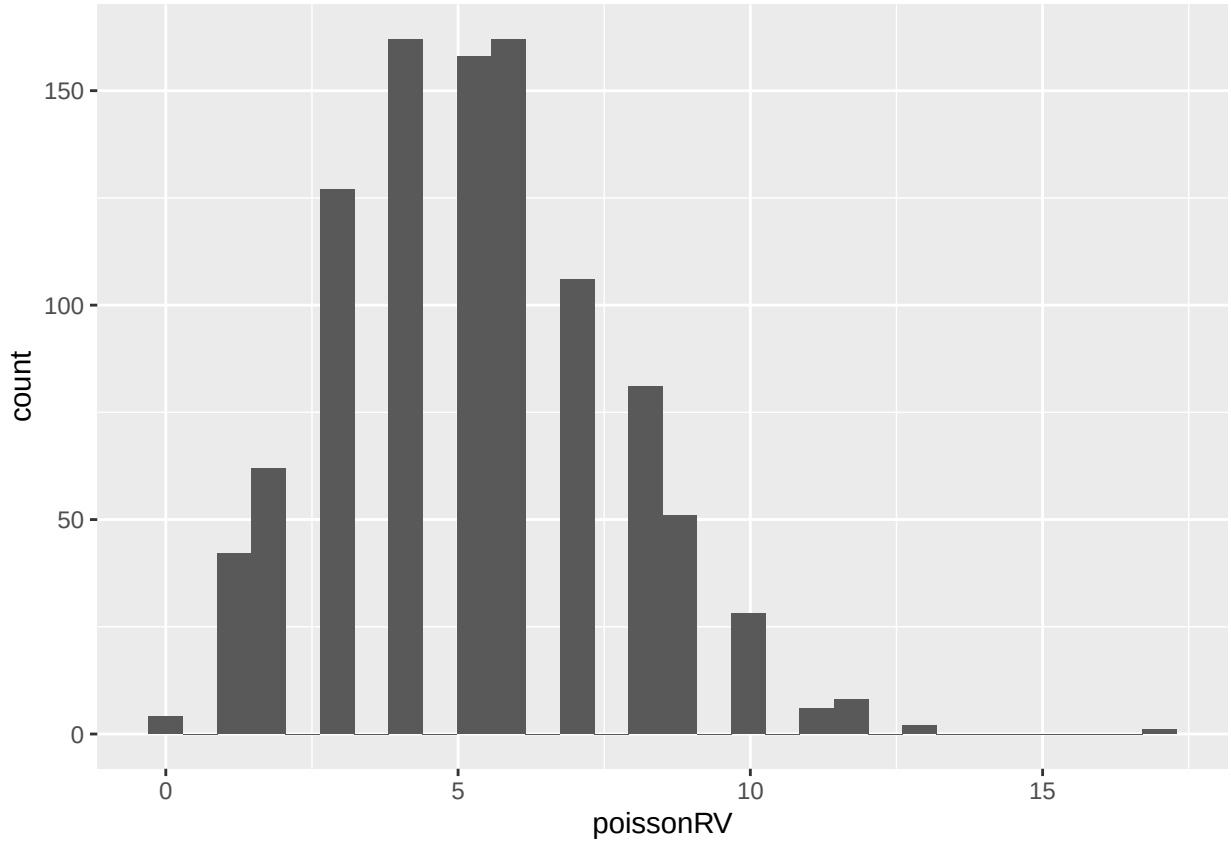
```
## # A tibble: 8 x 3
##   heads count   freq
##   <int> <int>  <dbl>
## 1     0    86 0.0086
## 2     1   512 0.0512
## 3     2  1613 0.161
## 4     3  2799 0.280
## 5     4  2732 0.273
## 6     5  1625 0.162
## 7     6   551 0.0551
## 8     7    82 0.0082
```

\end{enumerate}

**Exercise 11** For a poisson random variable with mean $\lambda = 5.2$. Use R to draw 1000 numbers from this distribution. Create a histogram of the results. What is the mean of the 1000 numbers generated? What is the variance?

```
poissonRV = rpois(n=1000,lambda=5.2)
results=data.frame(poissonRV)
ggplot(data=results,aes(x=poissonRV))+
  geom_histogram()
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```




```
print("The mean of the distribution is:")
```

```
## [1] "The mean of the distribution is:"
```

```
mean(poissonRV)
```

```
## [1] 5.291
```

```
print("The variance of the distribution is:")
```

```
## [1] "The variance of the distribution is:"
```

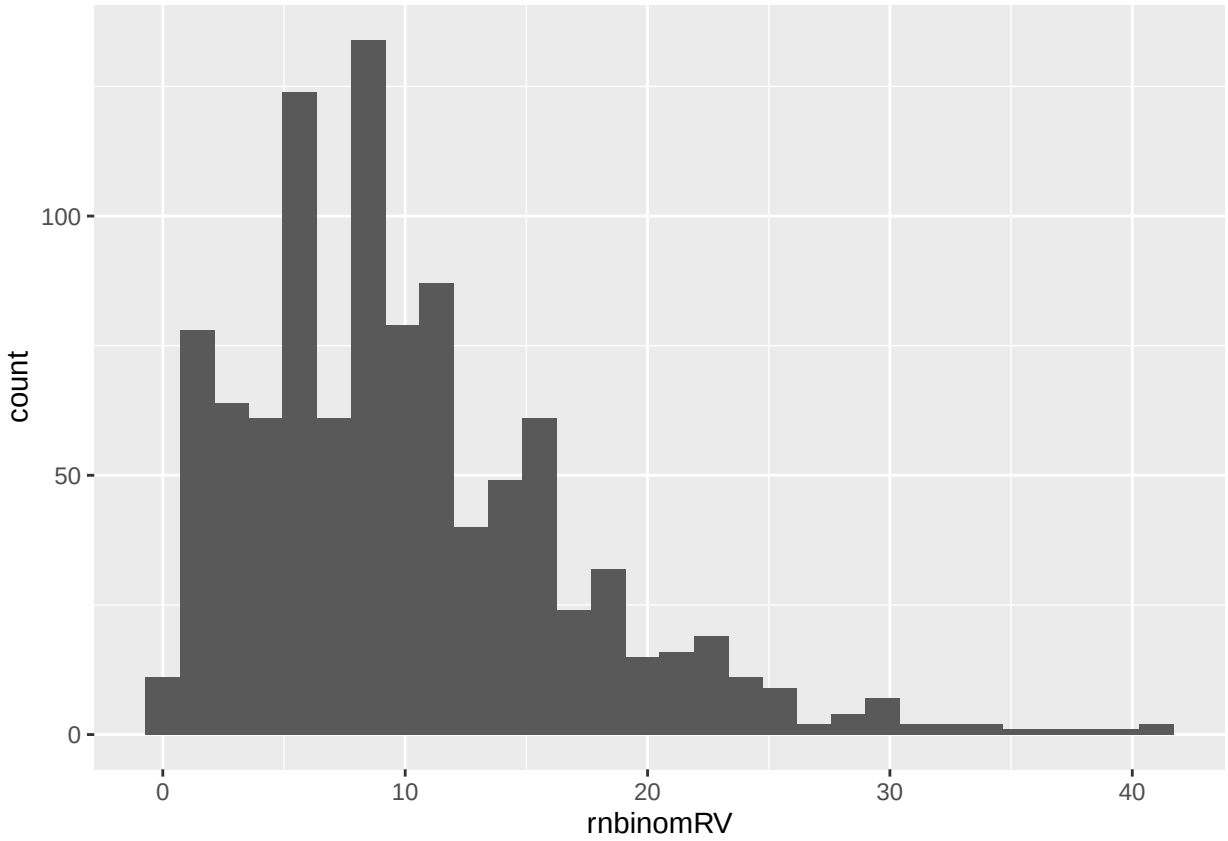
```
var(poissonRV)
```

```
## [1] 5.631951
```

**Exercise 16a** In R, generate 1000 draws of a negative binomial random variable with mean $\mu = 10$ and shape parameter $k = 3$. Plot the histogram of these results. Also compute the mean and variance of the 1000 results.

```
rnbinomRV = rnbinom(n=1000,mu=10,size=3)
results=data.frame(rnbinomRV)
ggplot(data=results,aes(x=rnbinomRV))+
  geom_histogram()
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```




```
print("The mean of the distribution is:")
```

```
## [1] "The mean of the distribution is:"
```

```
mean(rnbinomRV)
```

```
## [1] 10.096
```

```
print("The variance of the distribution is:")
```

```
## [1] "The variance of the distribution is:"
```

```
var(rnbinomRV)
```

```
## [1] 45.19198
```

**Exercise 27** Start with $S = 990$, $I = 10$, and $NewI = 0$ and assume that $\beta = 0.0024, \gamma = 0.05, delta.t = 0.2$. Create a for loop and use the si_step function defied above to model the population at each timestep for the first 1000 time steps. Plot the populations of $S, I$ $NewI$ through time. Now change Beta to be 10% of what it was.

```
si_step <- function(S, I, NewI, Beta, gamma, delta.t, ...){
  infections <- rbinom(n=1,size=S, prob=1-exp(-Beta*I*delta.t))
  recover <- rbinom(n=1,size=I, prob=1-exp(-gamma*delta.t))
```

```
  S <- S - infections + recover
  I <- I+ infections - recover
  NewI <- NewI + infections
  c(S=S, I=I, NewI=NewI)
}

#First we try running the function so that we know that it is working
#and the sort of output it produces.
si_step(S=990,I=10,NewI=0,Beta=0.0024, gamma=0.05,delta.t=0.2)
```

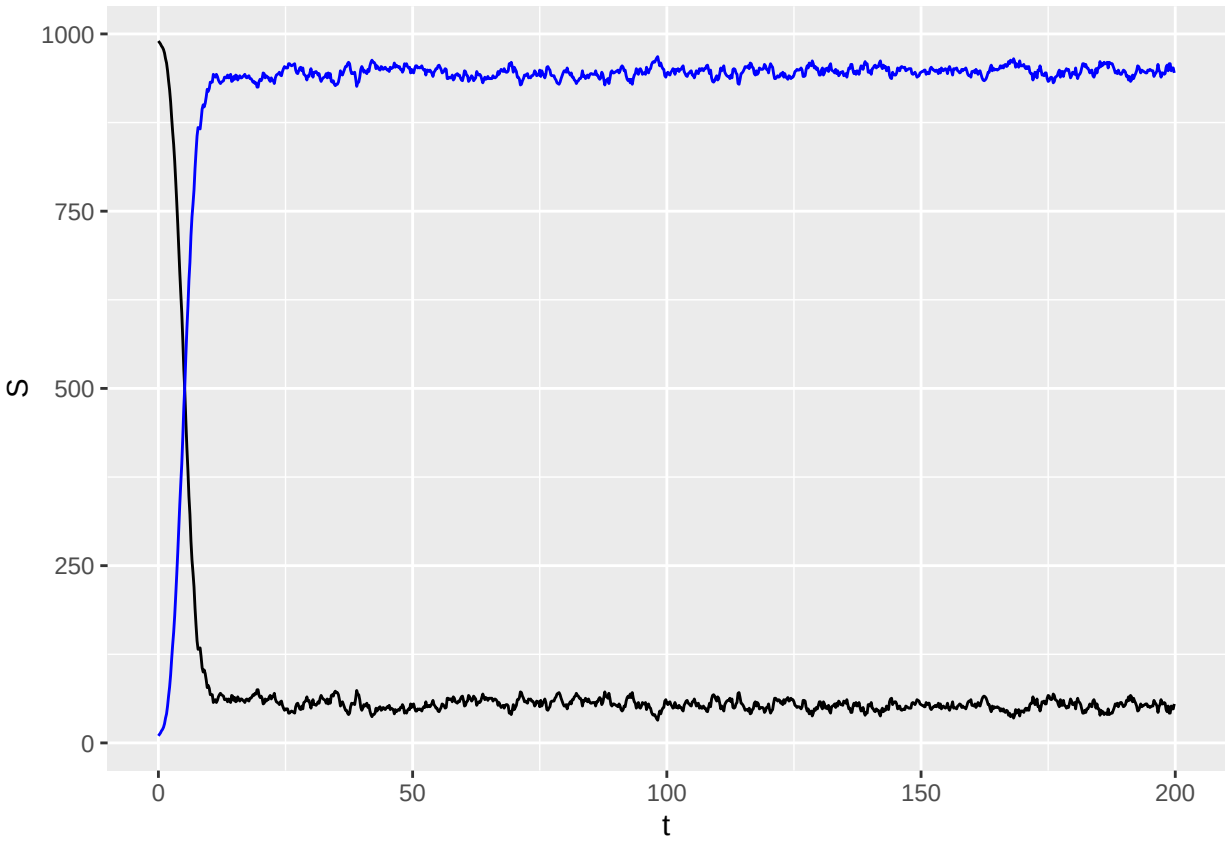
```
##    S    I NewI
##  985   15    5
```

```
#save the parameters we will use
Beta=0.001
gamma=0.05
delta.t=0.2

#Setup a data frame to store time and the populations
pop = data.frame(t=0,S=990,I=10,NewI=0)

#loop through the time
for(i in 2:1000){
  #compute and save the new populations and the new time
  pop[i,]=c(pop[i-1,"t"]+delta.t,
            si_step(pop[i-1,"S"],
                    pop[i-1,"I"],
                    pop[i-1,"NewI"],
                    Beta,gamma,delta.t))
}

ggplot(pop,aes(x=t))+
  geom_line(aes(y=S))+
  geom_line(aes(y=I),col="blue")
```




But if you run the above and look at NewI, you will see that it just keeps growing. It isn't actually tracking the new infections each day because we didn't reset it at the start of the day, it is actually tracking the total number of infections so far. To make it track the new infections each day, we would have to reset it to zero every day and only record the new values just before this. The goal of this exercise was to get you playing with the function a bit but getting that part to work right gets really fussy. Because we have the pomp functions, we don't have to figure out how to make it reset to zero, but you certainly can play with it if you would like!

**Exercise 31** One-Dimensional Slice: Set up a vector of 20 $\gamma$ values ranging from 0.0001 to 0.5. Use the same model as we have been using so far and set $\beta = 0.0024, \rho = 0.8, k = 5, N = 1000, \phi = 0.004$. For each value of $\gamma$, estimate the log-likelihood and error in the estimate. Make a graph of the log-likelihood values as a function of $\gamma$. For this set of parameters, what is the value of $\gamma$ that makes the model fit the data the best? Add 95% confidence intervals to your graph. Does the confidence interval change with $\gamma$?

Load in the needed packages and the observation data

```
library(doParallel)
library(doRNG)
library(readr) #this library is used to help load the csv file.
simuldata <-read_csv("https://raw.githubusercontent.com/kmontovan/pomp/refs/heads/main/SimulatedPompitis
```

```
## `curl` package not installed, falling back to using `url()`
## Rows: 150 Columns: 2
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## dbl (2): day, NewCases
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Setup the model

```
si_step <- function(S, I, NewI, Beta, gamma, delta.t, ...){
  infections <- rbinom(n=1,size=S, prob=1-exp(-Beta*I*delta.t))
  recover <- rbinom(n=1,size=I, prob=1-exp(-gamma*delta.t))
  S <- S - infections + recover
  I <- I+ infections - recover
  NewI <- NewI + infections
  c(S=S, I=I, NewI=NewI)
}

si_rinit <- function(N, phi, ...){
  c(S = round((1-phi)*N),
    I= round(phi*N), NewI=0)
}

si_rmeas <- function(NewI, rho,k,...){
  c(NewCases=rnbinom(n=1,size=k,
                     mu=rho*NewI))
}

si_dmeas <- function(log,NewCases, NewI, rho, k, ...){
  dnbinom(x=NewCases, size=k, mu=rho*NewI, log=log)
}

SI_Model <- pomp(
  data = simuldata,
    times="day",
    t0=0,
    rinit=si_rinit,
    rprocess=euler(si_step,delta.t= 1/4),
    rmeasure=si_rmeas,
    dmeasure=si_dmeas,
    statenames=c("S","I","NewI"),
    accumvars="NewI",
    paramnames=c("Beta","gamma","rho","k","N","phi")
  )
```

Set the values for gamma and assess the likelihood for the model with each of these values.

```
gamma_values = seq(.0001,.5,by=(.5-.0001)/19)
store_ll=numeric()
for(gamma in gamma_values){
foreach (i=1:10, .combine=c) %dopar% {
  library(pomp)
  SI_Model |>
    pfilter(
      params= c(N=1000, phi = 0.004,Beta=.0024, gamma=gamma, rho=.8, k=5 ),
      Np=500
      )
} -> pf
ll= logLik(pf)
store_ll = rbind(store_ll,c(gamma,logmeanexp(ll,se=TRUE,ess=FALSE)))
}
```

```
## Warning: executing %dopar% sequentially: no parallel backend registered
```

```r
plot_data=data.frame(store_ll)
names(plot_data)=c("gamma","ll_est","ll_se")

ggplot(plot_data,aes(x=gamma,y=ll_est))+
  geom_line()
```